

Introduction to SCIP

Computational Integer Optimization · SS 2026

Gennesaret Tjusila

What is SCIP?

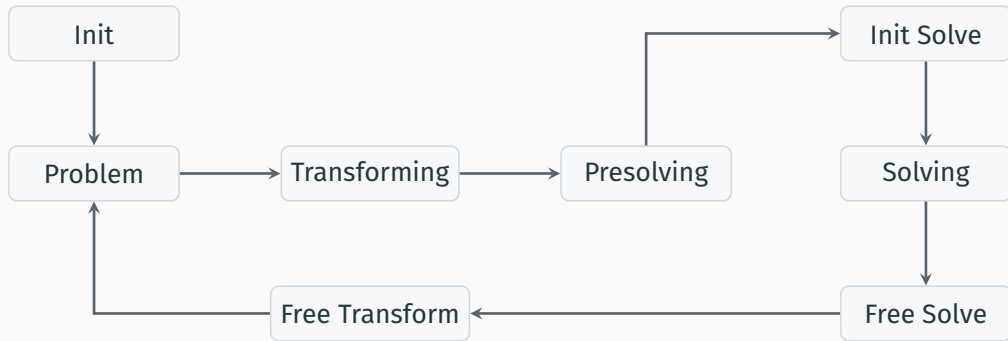
SCIP is a framework for constraint integer programming. It

- incorporates MIP features (cutting planes, LP relaxation), MINLP features (spatial branch-and-bound, NLPs), CP features (domain propagation), and SAT-solving features (conflict analysis, restarts);
- is a branch-cut-and-price framework with a modular plugin structure;
- is developed at the Zuse Institute Berlin; and
- is open source under the Apache 2.0 license.

Why SCIP?

- SCIP also supports Benders decomposition, exact MILP, IIS, MILP reoptimization, concurrent solving, and cumulative and logical constraints.
- Most importantly, SCIP is a platform for researchers to implement and test their own methods in a general-purpose solver.

Lifecycle of a SCIP solver instance



A tale of a farm

- We are a landlord who owns farmland.
- A library is like a rice farmer whom we employ.
- We ask the farmer to choose suitable land and create a rice paddy.
- The farmer reports success or failure.
- On success, the farmer writes the paddy's address on our blank plank.

Create the paddy — in C

```
#include "rice_farmer.h"

int main()
{
    RicePaddy* rice_farmer_paddy;

    RiceFarmerResponse t =
        RiceFarmerMakePaddy(&rice_farmer_paddy);

    if (t != RiceFarmerSaysEverythingOkay)
        return 1;
}
```

What does the farmer receive?

1. We create a variable where the farmer can write the address of the paddy.
2. We pass the *address of that variable* to the farmer:

```
RiceFarmerMakePaddy(&rice_farmer_paddy)
```

3. If successful, the farmer writes the paddy's address there.
4. We can then pass that pointer back whenever the farmer needs to find our paddy.

Water for seven days

- The farmer has many customers and does not remember where our paddy is.
- Each day, we pass the paddy pointer back as a reminder.
- The paddy is the farmer's: we neither know nor care what happens there.
- We should not touch it ourselves.
- If birds eat the paddy, the farmer must clean it up before we return an error.

Water the paddy — in C

```
for (int i = 0; i < 7; ++i)
{
    t = RiceFarmerWaterPaddy(rice_farmer_paddy);

    if (t != RiceFarmerSaysEverythingOkay)
    {
        RiceFarmerCleanUp(rice_farmer_paddy);
        return 1;
    }
}
```

Harvest and clean up

- On the seventh day, we are ready to harvest.
- We built the storage, so it is not a pointer.
- We give the farmer the address of our storage.
- Whether harvesting succeeds or fails, the farmer cleans up the paddy.

Harvest and clean up — in C

```
RiceStorage rice_storage;

t = RiceFarmerHarvest(
    rice_farmer_paddy, &rice_storage);

if (t != RiceFarmerSaysEverythingOkay)
{
    RiceFarmerCleanUp(rice_farmer_paddy);
    return 1;
}

RiceFarmerCleanUp(rice_farmer_paddy);
return 0;
```